Sum The File Report:

Code Explanation:

For this project I decided to use arrays for storing the numbers that is being read from the file and then at the end going through that array and summing up all of the numbers that are inside that array. For project we were allowed to use fopen, fscanf, printf, and fclose and so i utilized those in helping me with the project. I broke the code up into many different parts such such open_file, read_data, summing, and print_final. Inside of read_data we first start off by reading the first line of the file which indicates how many integers there are in the file and so we store this number as the count. Using fscanf we add the rest of the integers from the file into our array. After everything is added to the array we simply iterate through the array and add everything up and this step is done inside summing. After that is taken care of we use printf inside print_final to display the final sum that we got. So in my case using an array was the easiest way to add up all the numbers and kept everything simple.

Code:

```
section .data
   mode    db  "r", 0
   fmt_int db  "%d", 0
   sum_msg db  "Sum: %d", 10, 0
   extern fopen, fscanf, printf, fclose

section .bss
   array   resd 1000
   count   resd 1
   sum     resd 1
   fp      resd 1

section .text
global main

main:
   push    ebp
   mov     ebp, esp

   cmp     dword [ebp + 8], 2
   jne     exit

   mov     eax, [ebp + 12]
   mov     ebx, [eax + 4]


   push    mode
   push    ebx
   call    open_file
   add     esp, 8
   cmp     eax, 0
```

```
    je      exit

    call    read_data
    call    summing
    call    print_final

    push    dword [fp]
    call    fclose
    add     esp, 4

exit:
    mov     esp, ebp
    pop     ebp
    xor     eax, eax
    ret


open_file:
    push    ebp
    mov     ebp, esp

    push    dword [ebp + 12]
    push    dword [ebp + 8]
    call    fopen
    add     esp, 8

    mov     [fp], eax
    mov     esp, ebp
    pop     ebp
    ret

read_data:
    push    ebp
    mov     ebp, esp

    push    count
    push    fmt_int
    push    dword [fp]
    call    fscanf
    add     esp, 12

    xor     ecx, ecx
    mov     ebx, array

read_loop:
    cmp     ecx, [count]
    jge     done

    push    ebx
```

```
    push    fmt_int
    push    dword [fp]
    call    fscanf
    add     esp, 12

    cmp     eax, 1
    jne     done

    add     ebx, 4
    inc     ecx
    jmp     read_loop

done:
    mov     esp, ebp
    pop     ebp
    ret

summing:
    push    ebp
    mov     ebp, esp

    xor     eax, eax
    xor     ecx, ecx
    mov     ebx, array

sum_loop:
    cmp     ecx, [count]
    jge     .done
    add     eax, [ebx]
    add     ebx, 4
    inc     ecx
    jmp     sum_loop

.done:

    add     eax, [count]
    mov     [sum], eax
    mov     esp, ebp
    pop     ebp
    ret


print_final:
    push    ebp
    mov     ebp, esp

    push    dword [sum]
    push    sum_msg
    call    printf
```

```
    add     esp, 8

    mov     esp, ebp
    pop     ebp
    ret
```

Output:

```
[neevpl@linux4 ~]$ nasm -f elf32 reading.asm -o reading.o
[neevpl@linux4 ~]$ gcc -m32 reading.o -o reading
[neevpl@linux4 ~]$ ./reading randomInt100.txt
Sum: 4679
[neevpl@linux4 ~]$
```